



Autonomic Feature Selection using Computational Intelligence

Patrick Day^{a,*}, Stefano Iannucci^b, Ioana Banicescu^{a,b}

^a Center for Computational Sciences, Mississippi State University, Starkville, MS 39762, United States of America

^b Department of Computer Science and Engineering, Mississippi State University, Starkville, MS 39762, United States of America



ARTICLE INFO

Article history:

Received 1 August 2019

Received in revised form 9 November 2019

Accepted 8 April 2020

Available online 14 April 2020

Keywords:

Feature selection

Autonomic computing

Self-protecting systems

Computational intelligence

Deep learning

Representation & transfer learning

ABSTRACT

This paper introduces an autonomic method to optimize Feature Selection (FS) in autonomic systems while also presenting a taxonomy of FS techniques. Feature selection is a dimension reduction technique that has been proven to lead to improved performance by avoiding overfitting and to address complexity, thus providing faster and cost-effective algorithms. To be successful, the current FS methods are heavily reliant on two key elements: (1) a well defined and static learning objective, and (2) a relevant dataset. Current FS approaches involve mostly a manual process and do not take into account the constant change in the state of the underlying system. However, the method to automate the FS process presented in this paper, the Autonomic Feature Selection (AFS), uses representation and transfer learning concepts borrowed from the deep learning area. Representations of various types of monitoring data from a system are learned, and the resulting knowledge is transferred and re-used. The AFS is targeted to self-aware software systems, where the learning objective and (or) the availability of the relevant data are subject to change. The results suggest that although building and transferring learned representations is computationally expensive, its benefits outweigh the cost, by offering the potential to build flexible and re-usable knowledge structures.

© 2020 Elsevier B.V. All rights reserved.

1. Introduction

Self-Aware computing systems are systems that exhibit self-regulating behavior, and are dependent on the interpretation based on the particular information from their specific environments. The amount of data produced by complex systems are superabundant and therefore, it must be appropriately reduced to provide useful insights [1,2]. This reduction process is often referred to as Feature Selection (FS). An algorithm is only as good as the data with which it is provided, and therefore, careful consideration must be given to the algorithm for the selection of the essential features that make gaining any insight possible. The benefits of performing FS are significant, because they lead to improved algorithm performance by avoiding a common problem known as *overfitting* [3]. The overfitting occurs when the algorithm learns to exactly mimic the data, thus causing poor generalization of new data instances. Furthermore, the reduced complexity leads to faster and cost-effective algorithms.

In the literature, when machine learning is used, the FS is treated as a critical preprocessing step, and it is conducted as a manual task [4,5], which is both arduous and time-consuming [6,7]. The industry has accepted the need of FS as a necessary

form of data analysis. However, as new systems become increasingly complex and dynamic, new challenges emerge for FS. J. Li et al. [8], outline scalability, stability, and data streaming among such challenges. In particular, data streaming poses a significant problem when dealing with self-aware systems, because in order to gain insights for making decisions about how to effectively update the system, these systems require the capability to process large amounts of data in real-time. For these reasons, there is a need for a FS strategy that can meet these demands of self-aware software systems.

The current FS methods are heavily reliant on two key elements to be successful: (1) a well defined and static learning objective, and (2) a relevant dataset. These techniques assume a static target concept and do not take into account the constant change in the state of the underlying system. Thus, by following this strict approach, they provide a rigid methodology for identifying relevant features. In each case, sampled data are used to determine which features become inputs to a learning algorithm, and no consideration is given to the changes occurring to the underlying system. If changes happen, the process must begin anew. This strategy exposes new issues with intelligent systems. For instance, in Autonomic Computing [9], there is no guarantee that either of these prerequisites will be satisfied at the time needed. The system needs the flexibility to adjust its learning goals and, as the target changes, so shall the required selected features of the data. For this reason, it is far better to allow the

* Corresponding author.

E-mail addresses: pd560@msstate.edu (P. Day), stefano@cse.msstate.edu (S. Iannucci), ioana@cse.msstate.edu (I. Banicescu).

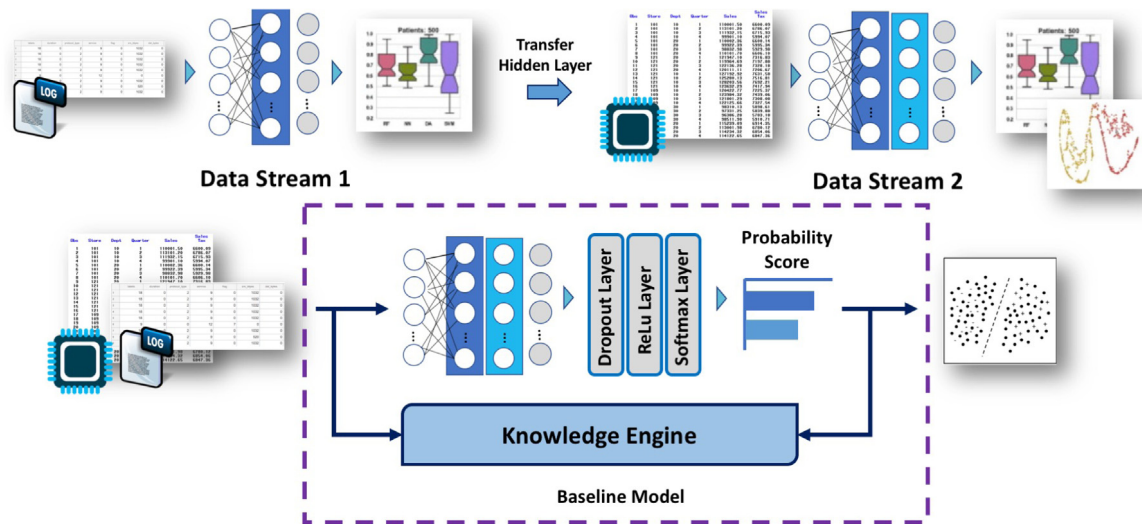


Fig. 1. Semi-supervised learning process.

system to determine itself the features that are most relevant to its learning goal at that moment.

There are two specific contributions presented in this paper. The first contribution is represented by a detailed taxonomy of the traditional feature selection techniques. The second contribution proposes a novel method, named 'Autonomic Feature Selection' (AFS), to address the limitations inherent in other feature selection methods. The taxonomy takes the three major feature selection classifications, namely, filter, wrapper and embedded, and breaks them down into sub-classifications. This taxonomy is useful because it provides a method to sort and define specific techniques, and also determines a way to apply them. On the other hand, the AFS extends an existing framework for the realization of self-protecting systems, named Autonomic Security Management System (ASMS) [10,11]. The functionality of an ASMS is meant to represent an improvement over the functionality of a current stand-alone Intrusion Response System (IRS), which basically performs a static mapping between response actions and specific attacks, therefore providing short-term rigid responses.

Although AFS is thought to be a component of ASMS, it has been developed as a module that can be integrated in any other solution aimed at providing self-protection. The proposed solution describes a two-step process: (1) a deployment of a model to learn the baseline, *normal* behavior, of a system component, and (2) an evaluation of the activity of that component and a report of any features that deviate from that model. The data streams produced by the system components, which are operational information mined from different sub-areas, are used to create the baseline model by utilizing a semi-supervised learning approach called deep learning [12,13]. This approach is used to build a representation of each stream using unsupervised learning, and then to transfer that knowledge to another model where it is trained using supervised learning. A knowledge engine is used in conjunction with the baseline model to accumulate any learned insights and provide system feedback. An example process is depicted in Fig. 1, where a neural network consisting of a deep hidden layer is used to learn the representation of the first data stream, which is a system log file. The weights of the hidden layer are then frozen, and the layer is transferred to a new neural network. The latter includes a second hidden layer that is trained with a data stream containing CPU and memory usage. The trained hidden layers of the two data streams constitute the baseline representation. The weights of both layers are frozen

and transferred again to another neural network, where three additional layers are added: (1) a dropout layer; (2) a dense convolution layer with Rectified Linear Unit (ReLU) activation; and (3) a dense convolution layer with Softmax activation. The new model uses supervised learning, and trains on labeled data to assign a probability score between two functionality classes, 'normal' or 'under attack'. The knowledge engine monitors both, the input and output of the model. It creates a mapping of the incoming features to one of the two classes, and stores the result. The *baseline model* results from a combination of the *baseline representation* and the mapping created by the *knowledge engine*. The baseline model is then re-trained at regular intervals by using the previous baseline model as a starting point. The use of pre-trained layers and of transferring that knowledge to new models decreases the overall training time. Any features that deviate from the baseline model are reported to the manager and compressed into a single unified stream via a union set.

This paper is organized as follows. In Section 2 a description is given on the background of feature selection and a taxonomy of FS techniques is provided. In Section 3 a detailed discussion of the Autonomic Feature Selection method is presented. In Sections 4 and 5 experimental results are presented with corresponding observations. A discussion of future work is provided in Section 6, and conclusions are highlighted in Section 7.

2. Background

Feature Selection is a class of techniques used in the reduction of the number of dimensions which a learning model is required to learn. In machine learning, the more dimensions or features that are present in a dataset, the higher becomes the degree of the function that is required to learn the target concept and therefore, more data points are needed. This phenomenon is known as the "Curse of Dimensionality", coined by Richard Bellman [1]. The benefits of performing FS are significant, since it leads to an improved performance model by avoiding overfitting, and to less complexity, which in turn leads to faster and more cost-effective models. Furthermore, it exposes the underlying processes which generated the data. Commonly used techniques, such as the Principle Component Analysis (PCA) [2], seek to map higher dimensional data into a lower dimensional space. The PCA re-builds a new set of features based on the existing variance in the data from the original set of features.

Autonomic computing is a paradigm derived from and modeled after the body's Autonomic Nervous System (ANS). The ANS

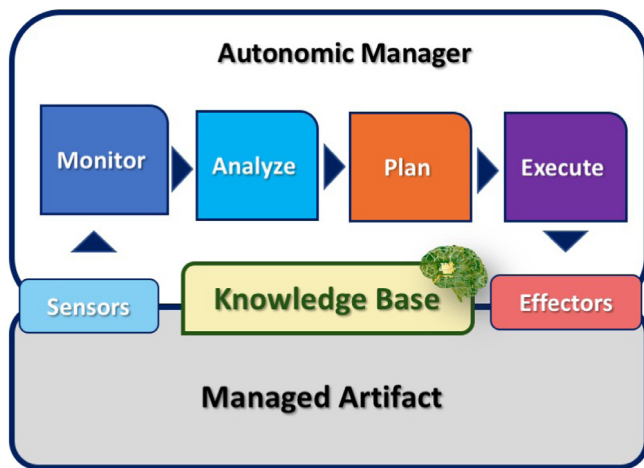


Fig. 2. MAPE-K feedback loop.

is the system that effectively controls all of the automatic functions of the body, such as: the heartbeat, breathing, temperature regulation, and all the other functions that happen without conscious thought. Similarly, autonomic computing systems exhibit self-regulating behavior, which may include: self-configuration, self-optimization, self-healing, self-recovering, or any other self-aware functionality [14]. These systems are also commonly referred to as “self-” systems. The autonomic computing reference architecture specified in [14], and further refined in [9], introduces an *autonomic manager* as the administrator of a *managed artifact*. These two units constitute an autonomic system. For brevity, going forward, an autonomic manager will be referred to as the “manager”, and the managed artifact will be likewise referred to as the “artifact”. An artifact can be, for instance, a software component, embedded hardware, or an industrial system. In an autonomic system, the manager is responsible for implementing any specified autonomic behavior. It performs this action by incorporating sensors and effectors, also known as touch-points. The sensors monitor the state of the artifact, while the manager determines what action(s), if any, should be taken, and the effectors carry out the necessary changes within the artifact. The manager utilizes what is known as the MAPE-K feedback loop, which stands for Monitor, Analyze, Plan, Execute, and Knowledge, as seen in Fig. 2. In the ‘Monitor’ phase, data is collected from monitoring devices and evaluated by the manager. These data are subsequently either stored for later analysis, or fed as data streams to the ‘Analyze’ phase. Example of data are: time series of performance metrics of a system (e.g., CPU load, memory usage, network throughput, and so on), alerts collected from Intrusion Detection Systems (IDSs), log files of systems and applications. The task of the ‘Analyze’ phase is to select the relevant features among all the input data streams, to correlate and merge them in order to produce a single added value data flow, and ultimately, to identify the current state of the system, which is then fed into the ‘Plan’ phase. Its task is to compute the response policy needed to drive the artifact towards the target state. Specific actuators on the artifact are in charge of applying the computed response policy, thus implementing the ‘Execute’ phase of the MAPE-K loop [10]. Finally, the ‘Knowledge’ layer serves as an interface between the artifact and manager which, during regular operation and over time, acquires insights about the artifact.

Computational Intelligence (CI) is another paradigm that is inspired from biological systems. The foundations of CI include Artificial Neural Networks (ANN), Evolutionary Computation (EC),

Fuzzy Systems alternatively referred to as Fuzzy Logic (FL), Swarm Intelligence (SI), Artificial Immune Systems (AIS), and others. A new category called *Soft Computing* is a combination of these techniques [15]. The ANNs model the biological neural systems; the EC models the natural evolution, including both genetic and behavioral evolution; the SI models the social behavior of organisms living in swarms or colonies; the AIS models the human immune system. Finally, the FL originates from studies of how organisms interact with their environment. The FL uses knowledge representations that comprise a series of conditional statements, such as IF-THEN rules. Developing such rule sets is expensive in time and intellectual capital. Therefore, to determine which rules to construct, combining fuzzy logic with neural networks or genetic computation is much more attractive.

Beyond both drawing inspirations from nature, autonomic computing and CI have more common points, such as knowledge and knowledge representation. Determining an appropriate method to represent data or knowledge is critical to effectively perform a learning task. Moreover, manually constructing a knowledge representation is difficult and time-consuming, which is a significant factor in the rise of Soft Computing.

Both Autonomic Computing and CI applications can have elements of Machine Learning, so they could also benefit from FS to reduce the set of features monitored in an artifact. Feature Selection in the current literature remains largely as a manual process, relying on domain knowledge and heuristic searches. This process contains four stages; (1) a generation procedure, (2) an evaluation function, (3) a stopping criteria, and (4) a validation procedure, as outlined in [4]. The authors of [7,16,17] present general surveys of FS and delve into varying levels of detail. In addition to this, in [7] a heuristic is also defined in the form of a checklist for performing feature selection. The common thread is that they specify three major classifications, named Filter, Wrapper, and Embedded.

2.1. Taxonomy of feature selection methods

Although the research around FS and Feature Extraction seems to be extensive, there is none or little information in the literature about the automation of this process. In this section, we present a taxonomy of FS techniques. The definition of a taxonomy for FS started with the organization of the techniques found in the literature. We decomposed the three major classes of FS, namely *filter*, *wrapper*, and *embedded*, as illustrated in Fig. 3. The framework presented in [4] was also taken into consideration. The authors categorized 32 different FS methods based on *generation procedure* and *evaluation function*. The three main FS classifications and sub-classifications types of the taxonomy align with the framework’s generations procedures and evaluation functions, respectively.

2.1.1. Filter classifications

The filter classification refers to a class of algorithms that calculates the variance of each feature and then selects those with the highest value. Filter algorithms are also typically faster, more scalable, and have lower computational complexity than wrapper algorithms. However, they ignore both feature dependencies and the interaction with the model. These methods emphasize removing a feature from a global set of features that do not meet some fitness criteria. Examples of filter methods techniques are [5,18–20]. According to L. Yu et al. [5], these selection algorithms are further categorized into two groups, namely *Feature Weighting* and *Subset Search*. However, these subgroups can be generalized as *Statistical Analysis (Feature Weighting)* and *Mathematical Programming*. Both of these subgroups use subset search algorithms to search through the global feature set to identify candidate subsets that are highly influenced by the correlation found between the feature and the predicted class.

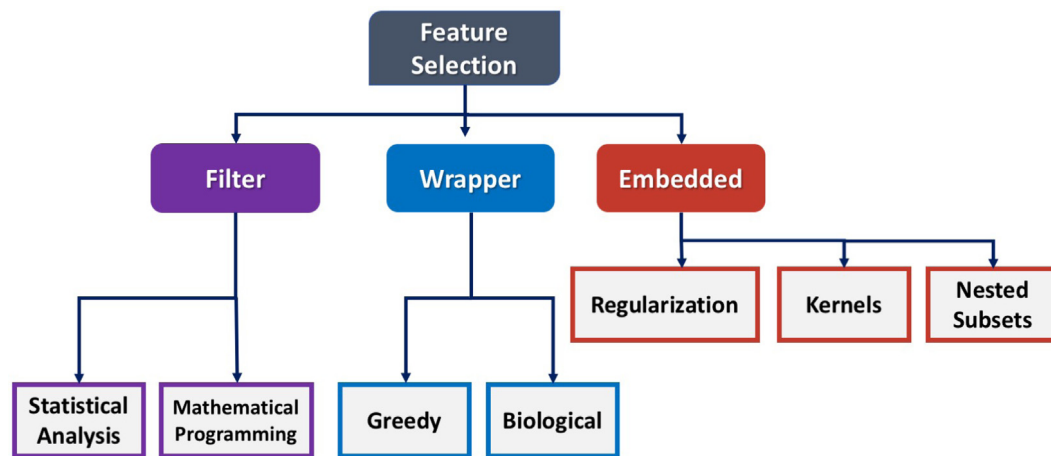


Fig. 3. Taxonomy of feature selection methods.

The majority of the filter methods heavily uses **Statistical Analysis** to assign a fitness value to features, based on their importance or relevance to a target concept. I. Guyon et al. [7] describe methods for feature ranking. Feature ranking is accomplished independently from the learning model, as a preprocessing step. It makes use of a scoring criteria which lists features in decreasing order and removes features with the lowest scores. Other statistical methods can be used instead of a scoring function, such as Single Variable. The classification [5] is focused on selecting features based on their predictive power. A variant of the Information Gain, called Information Theoretic Ranking [7], can also be used. Other examples include the ones described in [19] and [6]. The former presents a generalized Fisher score to select features jointly. The goal is finding a subset of features, which maximize the lower bound of the traditional Fisher score, while the latter proposes an algorithm named “Relief”, which uses statistical methods to avoid doing a complete search through the feature space. Relief focuses on highly correlated features, and thus, it is not optimal because it does not remove redundant features. In [18], a Chi-Squared test is used by the authors to improve the performance of five investigated classifiers. The Chi-Squared test is a statistical hypothesis test where the sampling distribution of the test statistic is a chi-squared distribution when the null hypothesis is true [21].

Mathematical Programming methods guide feature selection by converging on a subset of features that linearly correlates to the target concept; similarly to a Linear Programming problem to solve a system of equations subjected to many constraints, [20] takes the feature selection process and converts it into a logistical regression classification for each feature. The authors formulate the problem as a mathematical programming problem with a parametric objective function. In order to minimize the misclassified points, a separate decision plane is generated in the feature space with the smallest dimensions possible, while minimizing the average distance of misclassified points on the plane. The selected features are an optimal set that can approximate the target concept. As mentioned, [19] maximizes the lower bound of the traditional Fisher score, but the underlying feature selection is based on a mixed integer programming problem, which the authors reformulate as a quadratically constrained linear programming problem.

2.1.2. Wrapper classifications

The algorithms that belong to the wrapper class attempt to find an optimal subset of features by iteratively evaluating the performance of the classifier, continuously updating the subset and re-evaluating the model using cross-validation. The process

continues until a subset of features has visible improvement over the performance of the preferred learning model. This strategy involves re-training and re-testing a specific model, therefore restricting the search to a certain type of algorithm. Wrapper approaches combine the interaction between both, the selected features and selected model, while also accounting for dependencies among the features. A typical disadvantage of these techniques is that they are computationally intensive and prone to overfitting.

Although these methods are brute force solutions with extensive searches through the solution space, their coarse search strategies can curtail issues with overfitting, as demonstrated in [3]. The techniques using this method include the ones described in [22–26]. In most cases, the preferred learning model is treated as a black box to evaluate subsets of features according to their performance. Wrapper techniques cover a wide range of FS and can be difficult to categorize, due to their diversity and the way they can be combined. Nonetheless, we identified the following subclasses: *Greedy* (forward selection and backward elimination), *Biological*, and *Raw Extraction*.

Greedy search strategies seek to optimize a solution by choosing the optimal option in the short-term, without regard for long-term consequences. The characteristics of these methods are that they are efficient, easy to implement, and robust against overfitting. Often they do not lead to a globally optimal solution, while providing reasonable approximations of the global optima. Since the greedy approach is a well known algorithm design strategy, many types of algorithms exhibit the greedy behavioral characteristics. For instance, Genetic Algorithms (GA) implement a biologically inspired optimization search, based on the Darwinian principle of the ‘survival of the fittest’. The GA obtain an optimal or near-optimal solution after a series of iterative computations. Because of the penchant of performing well on optimization searches, the tendency is to use GA in conjunction with other methods. In [26], the authors provide an example combining a Decision Tree with a GA approach to select a subset of input features for a decision tree classifier. In general, Decision Trees use some form of entropy (information theory) to select features to become nodes of the tree. This process is handled greedily by choosing the feature with the highest level of correlation at each stage. The goal is to increase the detection rate and to decrease the false alarm rate in network intrusion detection. To illustrate another example of an inherently greedy approach crossing over with another category, [25] uses a Branch and Bound (BnB) algorithm to generate an efficient enumeration scheme. BnB conceptually can be viewed as an amalgam of both, a mathematical programming problem and a decision tree problem. The solution space is first created with all possible configurations,

and then pruned to a feasible set of solutions based on the given constraints. The solution space is branched and recursively searched, checking whether the bound of the current node is higher than the upper bound for some previously visited node.

Biological methods largely draw inspiration from the study of biological systems such as the previously discussed ANNs and GAs. ANNs are computing systems that model the brain and the nervous system. Such networks learn to perform tasks by considering examples, and then infer meaning about unseen data samples. Learning, in this sense, is identifying the correlation between the features in the samples and how that relationship impacts the target concept. Often, in the literature, ANN and GA are employed together, where the GA uses a neural network as the classifier to either explicitly reduce the dimensionality by optimizing weight selection, or network configuration. In the case of the latter two, the feature selection is performed implicitly. Examples are included in [27–29]. A special case of Biological methods, called *Raw Extraction*, utilizes Deep Learning techniques which emphasizes learning data representations, contrary to other ML methods that focus on a specific learning objective [30]. These methods focus on extracting features indirectly from raw data, such as image or audio files. For example, the representation of an image can be learned by using autoencoders to combine a set of simpler concepts, such as corners and contours, which define edges [31]. Y. Bengio et al. [32], suggest that deep architectures, known as Deep Belief Networks (DBN), can be used to learn complicated functions that represent high-level abstractions. The authors of [33] used DBN to extract features from audio-visual data to construct an emotion recognition system. Other examples utilizing raw extraction include the ones in [34], where geometric features are extracted from Computer-Aided-Design models using rule-based pattern recognition applied to patterns stored in a knowledge base. The paper written by F. Muge et al. [35] uses digital image analysis algorithms regarding filtering, segmentation, separation of text from non-text, and word recognition to generate a set of features.

2.1.3. Embedded classifications

In general, embedded feature selection algorithms lean toward features that will reduce the complexity, therefore, the amount of overfitting and variance of a model as well. A penalty or bias is applied to features that are noisy or having too much variance. Similarly to wrapper methods, embedded algorithms are geared toward a specific learning model, and have the added advantage of including the interaction with the learning model. Compared to wrapper techniques, these methods tend to be more cost-effective and make efficient use of the available data by not needing to separate training and validation sets. Therefore, they converge to a solution faster and are not as computationally intensive. Examples of this classification are given in [26,36]. Embedded routines are composed of the following subcategories: *Regularization*, *Kernels*, and *Nested Subsets*.

Regularization can be used to enforce sparsity constraints on a set of features, which in turn leads to more coherent learned models in practical applications. A sensible sparsity constraint is the L_0 norm $\|w\|_0$, defined as the number of non-zero elements in w . Solving a L_0 regularized learning problem, however, has been demonstrated to be NP-hard. The L_1 norm can be used to approximate the optimal L_0 norm via convex relaxation. It is proven that the L_1 norm induces sparsity. There are multiple forms of regularization, each having specific characteristics, such as, Forward Stepwise, Best-subset, Ridge Regression and, most relevant to FS, the Least Absolute Shrinkage and Selection Operator (LASSO) [37]. LASSO is a regression analysis method that performs both variable selection and regularization to enhance the prediction accuracy and interpretability of the statistical model it produces.

Trevor Hastie et al. [37] also discuss **Kernels**. Kernel methods are a class of algorithms for pattern analysis, mostly utilized in classification problems, in particular, Support Vector Machines. Kernel methods use kernel functions to operate in high-dimensional feature space without computing the coordinates of the data. Instead, these methods compute the inner products between images of all pairs in the feature space. This operation is often computationally cheaper than the explicit computation of the coordinates. Kernel functions are also a good way to encode domain-specific knowledge into a learning model. This approach is called the *kernel trick*. The kernel trick avoids the explicit mapping that is needed to let linear learning algorithms learn a nonlinear function or decision boundary.

Nested Subsets Some embedded methods guide their search by estimating changes in the objective function value incurred by making moves in feature subset space. I. Guyon et al. [7] discuss nested subsets and specify three different distinctions: (1) finite difference calculation, (2) quadratic approximation of the cost function, and (3) sensitivity of the objective function calculation. The first two methods rely on efficient computations based on the least squared model, by using forward variable selection, essentially adding the feature that most decreases the Mean Squared Error (MSE). The latter also requires the elimination of features with the smallest impact on the MSE. However, the sensitivity to changes in the feature coefficients is used to devise a forward selection procedure. Perkins et al. [38] presents this approach, and the application involved uses a linear model with a cross-entropy objective function.

3. Autonomic feature selection

The literature in the area of Deep Learning research has increased in recent years and has made considerable advances in the areas of computer vision, speech recognition, and natural language processing. Deep Learning techniques, which emphasize on learning data representations, are composed of ANNs with multiple layers to extract high-level features from input data. The activity of extracting features in this manner is known as Representation Learning. According to the literature, what makes the representation learning such a powerful concept is that, regardless of how it is obtained, once learned, it is *reusable* and *transferable* to different learning models [30,31]. The process of transferring the learned representation from one domain to a slightly related, however different domain is known as Transfer Learning [12,13]. Another key advantage of using transfer learning is that it decreases the training time when using pre-trained networks, while simultaneously decreasing the amount of data required for learning.

The approach proposed in this paper builds on the concepts of Representational and Transfer Learning. Furthermore, the application described in this section using these principles is novel in its approach because, while most of the work in the literature focuses on applications, such as text & image recognition [35,39], audio-visual [40–42] and natural language processing [43–45], the data type used here is time series based.

The proposed autonomic approach is characterized by a sequence of two tasks: (1) learn and determine the baseline of the *normal* behavior of the artifact; (2) learn to distinguish and report the abnormal behavior when the artifact is under attack.

3.1. Establishing a baseline

The system managed by the ASMS could have hundreds of thousands of varying types of artifacts. For example, an artifact could be a Linux server running critical applications, an Android mobile device ingesting IoT data, or an enterprise industrial asset.

Each artifact could have a large number of data sources to learn, with each source having several features.

In order to learn the baseline, we use machine learning algorithms. The latter need data, and the chosen representation of that data has an impact on the computational complexity of the training phase. A server running a Linux operating system, for instance, will monitor, among others, its filesystem, firewall, active ports, logs, memory, CPU, and network traffic. An android device will instead work on a different set of features. Choosing an appropriate representation is key to a successful model. Encoding is defined as learning representations of the data that makes it easier to extract useful information [30]. Therefore, instead of constructing a representation by hand, we allow the artifact to learn it by itself. Ultimately, this is a two-stage process: first, a representation is built from the data streams using data collected under normal operation. Afterwards, this knowledge is transferred to a new classification model trained using data collected while the artifact is under attack. The result of the process is a model that is an accurate baseline of the artifact that is ready to be deployed as an active monitor.

3.1.1. Long short-term memory neural networks

Existing monitors residing within the ASMS network produce data that is based on time series. It is composed by a series of events indexed in time order. Recurrent Neural Networks (RNNs) are a family of neural networks for processing sequential data such as time series [46]. What makes RNNs especially suited for processing sequences is the ability to save the weighted output of previously seen input, and then feeding that information back into the input. Recurrent networks have two sources of input: the present and the recent past, which are combined to determine how the network responds to new data, providing the ability to simulate a time delay in the data. It is worth noting that indexed time steps do not have to necessarily refer to time, because they can simply refer to a position in a sequence. The decision a recurrent network produces at time step $t - 1$ affects the decision taken one moment later at time step t . I. Goodfellow et al. [31] consider equating a recurrent network to a computational graph.

Long Short-Term Memory (LSTM) neural networks, also referred to as *gated* RNNs, are considered the most effective type of network for modeling sequences according to the literature [31–33]. Gated RNNs derive from the notion that the rate of change at each time step could either be zero, or grow exponentially, due to the use of connection weights. These connection weights allow the network to acquire knowledge over time in each node, and then provide, when needed, a mechanism to forget the information by setting the weight to zero. Therefore LSTM neural networks not only have the characteristics of a computational graph to provide feedback, but also provide correction weights on the information feeding back into the network, which allows the network to learn to decide when to memorize or forget learned information. This ability for memorization is the reason LSTM networks are the most effective for modeling sequences and allowing increased capacity for the time scale or sequence length as a direct result.

3.1.2. Learning the baseline

An autoencoder is a model that has a straightforward objective function, which is to learn how to replicate the input to the output. Ordinarily, it seems pointless to copy the input to the output nodes, but the composition of an autoencoder comprises: an encode function $f(x)$, a hidden layer h , and a decode function $g(h)$. It is not the output of the decoder that is important, but the hope that internally h takes on valuable properties while encoding the input, therefore making autoencoders the ideal model for learning the representation. The code in Listing 1 is a snippet that

```

1 from keras.models import Sequential
2 from keras.layers import LSTM, Dense, RepeatVector,
  ↳ TimeDistributed
3
4 # input shape [samples, timesteps, features]
5 steps = 16
6 data = data.reshape((145737, steps, 46))
7
8 # Encoding Section
9 model = Sequential()
10 model.add(LSTM(16, activation='relu', input_shape
  ↳ =(steps,46), name='data 1'))
11
12 # Decoding Section
13 model.add(RepeatVector(steps))
14 model.add(LSTM(6, activation='relu',
  ↳ return_sequences=True, name='decode'))
15 model.add(TimeDistributed(Dense(46)))
16 model.compile(optimizer='adam', loss='mse')
17
18 # fit model
19 model.fit(data, data, epochs=50)
20 model.save('saved_model.h5')
21

```

Listing 1: Defining Autoencoder for Representation Learning

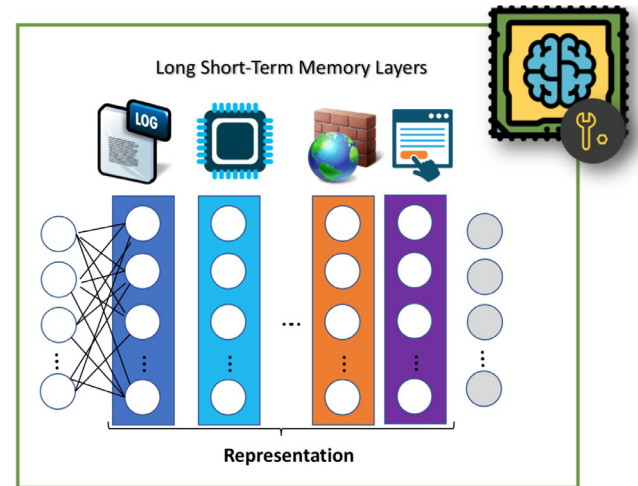


Fig. 4. Learning the representation from disparate data sources.

was used to describe the autoencoder used to learn the baseline representation. The data frame *data* in line 6 has a shape of 16 time-steps (mini batches), and 46 features for this data stream. The LSTM layer in line 10 is the encoder, which imposes a constraint by mapping the input into a smaller dimensional space. An autoencoder whose encoded dimensions are less than the input dimensions, in this case from 46 to 16, is called *undercomplete*. Learning an undercomplete representation forces the encoder to capture the most salient features of the training data [31].

The decoder section, by taking the output of the encoder, must reconstruct the input from the representation. The learning process entails the minimization of the loss function in Eq. (1).

$$L = \frac{1}{n} \sum_{i=1}^n (x - g(f(x)))^2 \quad (1)$$

where L is the Mean Squared Error (MSE) loss function, which penalizes $g(f(x))$ for being dissimilar from x . When using MSE with a linear decoder, the autoencoder learns the Principle Components subspace.

There is one major limitation when using representation learning, and that has to do with the input shape of the training data.

Once the process of training has begun, all subsequent hidden layers must have the same input shape, the number of time-steps (mini batches), and number of attributes. Shape uniformity is assured by training using the data stream with the most attributes first, and then padding the remaining streams to fit the same shape. Moreover, along the same vein, in order to ensure a useful representation while using an undercomplete encoder, each stream has to map to a dimensional space that is smaller than the stream with the smallest amount of attributes.

3.1.3. Assembling the baseline

In the case of the Linux server, all the data streams including the filesystem, firewall, active ports and others, describe a time series data set. A LSTM deep layer is used to learn the nuances of that data, creating a representation of that particular data stream. Once the hidden layer has learned the data from a stream, all the weights are frozen, preserving the gained knowledge. Thus, a new model is constructed with the transferred knowledge and another LSTM hidden layer is added to train a new data stream. The entire process is explained in the Algorithm 1. It is worth noting that data streams can be trained sequentially one at a time, as outlined in the algorithm, or concurrently, in order to collect and transfer all trained layers into an operational model. The result is a set of hidden layers, each one corresponding to a learned representation of separate data streams, as portrayed in Fig. 4. The learned representation signifies the normal behavior of the artifact and marks the completion of the first phase of assembling a baseline.

The power of representational learning lies in the fact that it does not matter how the representation is initially learned and therefore, several techniques can be utilized to obtain it. The initial phase requires the use of unsupervised learning techniques on large amounts of unlabeled datasets collected during normal operation of the artifact. The next step in baseline assembly is for the artifact to learn when it is under attack. The representation learned in the previous phase is transferred to another model with a learning task to classify if the artifact is either in an attack or healthy state. The second stage utilizes supervised learning, as labeled data is used to train the new model. The technique of using both unsupervised and supervised methods is called *Semi-Supervised* learning [31]. The new model introduces three additional layers to the configuration. The first new layer is a Dropout layer. Its sole purpose is to prevent overfitting. This layer takes a float value between 0 and 1 as an argument and randomly selects the percentage of weights to be set to zero during each update. The next layer to be added is a fully connected Dense layer with a Rectified Linear unit (ReLU) activation. Dense layers implement operation, $y = f(\sum_i^n x_i * K + x_0)$, where f and K are the activation and kernel functions, respectively. A kernel method is used to map data into a higher dimensional space, such that it can be linearly separated. Finally, the last layer to be added is another Dense layer with a Softmax instead of a ReLU activation function.

$$y_k = \frac{\exp(\phi_k)}{\sum_j^D \exp(\phi_j)}, \quad (2)$$

$$E_{\text{entropy}} = - \sum_n^c [t_k^n \ln y_k^n + (1 - t_k^n) \ln(1 - y_k^n)] \quad (3)$$

The Softmax function of Eq. (2) flattens a vector in the range (0, 1) and ensures all the resulting node outputs add up to 1. The output scores y_k are the probability scores of belonging to a specific class in D . Softmax cannot be applied independently to each y_k , because it depends on all outputs of y , where ϕ_j are the scores inferred by the gain for each class in D . It is worth noting that the Softmax activation for a class ϕ_k depends on all

Algorithm 1 Autonomic Baseline Learning

```

1: procedure LEARN BASELINE
2:   Initialize;
3:    $network \leftarrow empty$ 
4:    $model \leftarrow empty$ 
5:    $performance \leftarrow empty$ 
6:    $knowledge \leftarrow empty$ 
7:    $num \leftarrow num\_datasources$ 
8:    $streams \leftarrow COLLECT(num)$ 

9:   #Configure Network
10:   $network \leftarrow CONFIGURE()$ 
11:   $network \leftarrow ADD(network, lstm\_layer)$ 

12:  #Train Model
13:  while NEXT(streams) do
14:     $model \leftarrow TRAIN(network, streams[num - 1])$ 
15:     $performance \leftarrow EVALUATE(model)$ 

16:    if OK(performance) then
17:       $layer \leftarrow GET(model, lstm\_layer)$ 
18:       $knowledge \leftarrow ADD(knowledge, layer)$ 

19:    #Transfer Knowledge
20:     $network \leftarrow CLEAR(network)$ 
21:     $network \leftarrow ADD(network, knowledge)$ 
22:     $network \leftarrow ADD(network, lstm\_layer)$ 

23:  return knowledge

```

the scores in y . The variable E_{entropy} , referenced in Eq. (3), is the Cross-Entropy loss function for two classes, where E_{entropy} is the error due to entropy; t_k^n , previously observed probability, and y_k^n is the newly attained probability score. When cross-entropy is combined with the Softmax function, a Categorical Cross-Entropy loss function is produced. This function will train the model to output a probability over the C classes for each datum.

Ideally, learning the representation to create a baseline is part of the bootstrapping process of an artifact, because the activity is computationally expensive and time-consuming. An additional benefit of creating this representation up-front is that the re-training process becomes much less expensive by re-using the persisted representation as a starting point. The pre-trained layers require less time to train and converge much faster to a set of weights that account for the new data. The decrease in training time for new data is a significant benefit of transfer learning and, as a consequence, the longer the artifact is active, the less expensive becomes the re-training process. The latter is handled off-line and consists of three steps: (1) while the artifact is operational, additional processes are used to collect new data from all the data sources used to create the initial baseline; (2) each LSTM hidden layer is re-trained with the same process described by the Algorithm 1; (3) the updated representation is then re-deployed to the baseline model.

3.2. Deploying the baseline model

The completed baseline model deployed on an artifact acts as a monitor, with the ability to successfully produce a value between (0, 1) signifying the probability that the artifact is under attack.

However, there is no way to map the output directly to the input features, mainly because the representation creates new features from the original, therefore the model alone does not

work in terms of feature selection. What is needed is a way to map the output of the baseline back to the current state of the artifact, and to its contributing features. As a solution, a Knowledge Engine is coupled with the baseline model when deployed. The engine contains facts that constitute the knowledge-base of the system, which can be queried against. The engine provides updates to the fact-base if the proposition of the query is found in the knowledge-base, or if it is implied by knowledge-base; otherwise, a negative result is returned. The Python Knowledge Engine (Pyke), an existing library, is used to implement this functionality. Pyke integrates a form of Logic Programming into Python by providing both forward-chaining (data-driven) and backward-chaining (goal-directed) inferencing. Also, by attaching Python functions to backward-chaining rules, Pyke automatically generates programs by assembling individual functions into complete call graphs [47]. The knowledge engine observes the input to the baseline model and inspects the output, using a combination of forward-backward chaining rules to infer which set of features contribute to the observed attack state.

Once the baseline model and knowledge engine are successfully deployed and running on the artifact, for each new data sample, the engine takes notice of the incoming data stream, its features and their values, and adds that information to the fact base via forward-chaining rules. After that, backward-chaining rules invoke the baseline model on the saved facts to determine if the artifact is under attack, and the output is persisted back into the fact base for retrieval. If the model determines that an attack is occurring, the list of features that are supposed to pertain to the attack is communicated to the manager.

The manager will accumulate all of the reported features from each managed artifact and create a union set of those features. In Section 3.1 we provided examples of potential artifacts, a Linux server running critical applications; an Android mobile device ingesting IoT data; and an enterprise industrial asset. Fig. 5 gives more context to these examples, while visualizing the concept of how features are collected after being reported to the manager. The union set of features is a compression of all the data streams. Each artifact has different data streams and, therefore, unique features. Artifacts 1 and 2, Linux servers and mobile devices, respectively are both computing devices. Although different, both artifacts have a lot more in common than Artifact 3, which is a gas turbine (a SCADA system). For instance, let us suppose that feature Z is reading from a vibration sensor. If the feature is found to be interesting in Artifact 3, then the manager, from its perspective, will respond as if all artifacts reported a vibration reading. Similar to rules in a Network Access Control List, artifacts that do not support a vibration sensor will show a null value for that feature, and ignore any corresponding policy. The union set is meant to be a reduced feature set of the global system and provides a single view for the manager to evaluate the current state of the entire system, thus reducing the need of the manager to keep track of which feature came from which artifact. The manager can then focus on computing response policies for the system as a whole, instead of creating individual policies for each artifact.

4. Experimental results

The approach proposed in this paper, the Autonomic Feature Selection, was applied within the context of the Autonomic Security Management System. The specific use case entailed identifying the features corresponding to an attack state of an artifact, such that the manager could analyze and plan accordingly. Testing the validity of this solution required using an established public data set and simulating the activity of an artifact. To this end, we used the dataset UNSW-B15, created and benchmarked by N. Moustafa et al. [48,49]. These authors published

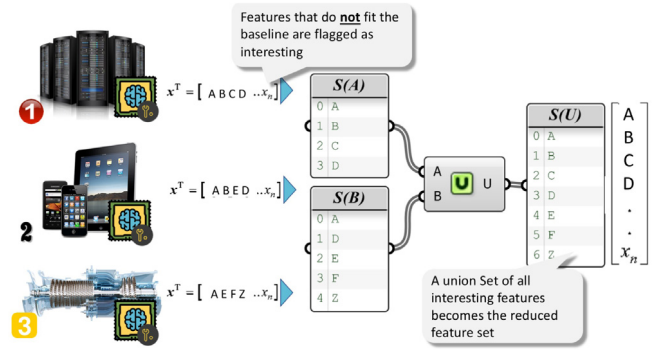


Fig. 5. Union set of all reported features from monitored artifacts.

a series of papers on Intrusion Detection, Network Forensics, Privacy-preserving, and threat intelligence approaches in different systems, such as, Networked Systems, Internet of Things (IoT), SCADA, Industrial IoT, and Industry 4.0, using the datasets reported in [50–53]. The raw network packets (PCAP files) of the UNSW-NB15 dataset are created by the IXIA PerfectStorm tool in the Cyber Range Lab of the Australian Centre for Cyber Security (ACCS) for generating a hybrid of real modern normal activities and synthetic contemporary attack activities. The content of UNSW-NB15 includes a number of raw source files, such as PCAP, BRO, and Argus files. Furthermore, the dataset contains CSV files, which are the result of Extract, Transform, and Load (ETL) processes, executed to process the binary files:

1. UNSW-NB15_features.csv –Feature explanations
2. UNSW-NB15_GT.csv –Ground Truth table
3. UNSW-NB15_LIST_EVENTS.csv –List of attacks
4. UNSW-NB15_(1–4).csv –Network traffic data
5. UNSW-NB15_training-set.csv –Training data
6. UNSW-NB15_testing-set.csv –Validation data

Ideally, different but related data types are used for AFS, such as CPU, memory, file system, and others. However, most of the public data sets on network intrusion focus on network traffic. For this reason, although the proposed approach is able to work with diverse data sources, we focus in this work only on network data. To completely realize the usefulness of this approach it is best to use two or more data streams to build a baseline representation.

We created two data streams. For the data stream (i), we used raw network data contained in the PCAP files referenced in Table 1. For the data stream (ii), we used instead the digested network logs contained in the UNSW-NB15_(1–4).csv files. Given that the latter is a superset of the raw source data files (PCAP, BRO, and Argus), we performed a minor extraction process to remove the PCAP features from it, in order to have two non-overlapping streams shown in Table 2.

4.1. Principle component analysis (PCA) comparison

To implement and validate the baseline representation, data streams (i) and (ii) were used. The data in data stream (ii) has 46 features, and therefore, as first step, we performed data analysis. The “Explained Variance Ratio” was calculated using PCA to identify how many dimensions were necessary to maintain 95% of the variance within the data, and the result is plotted in Fig. 6. The curve is the variance of each consecutive principle component until the value becomes greater than or equal to the threshold, $\sum_{i=1}^n x_i \geq 0.95$, where n is the number of features in the data set, and x_i is the variance of each principle component. The figure shows that the majority of the variance in the dataset is captured

Table 1
PCAP data example—data stream (i).

PCAP data					
Time	Source	Src_Mac_Add	Src_Port	Destination	Dest_Port
0.000000	175.45.176.3	MS-NLB-PhysServer-26_c5:00:00:00	22592	149.171.126.16	143
0.000041	175.45.176.3	Vmware_a5:24:c2	22592	149.171.126.16	143
0.078886	149.171.126.16	PaloAlto_05:9e:1c	143	175.45.176.3	22592
0.078896	149.171.126.16	Vmware_a5:77:63	143	175.45.176.3	22592
0.101585	175.45.176.0	62762	MS-NLB-PhysServer-26_c5:00:00:00	149.171.126.16	56430
Protocol	DSCP	Length	Bytes	Severity	Info
IMAP	CS0	77	77	null	Request: a003 SELECT INBOX
TCP	CS0	77	154	Note	[TCP Retransmission] 22592 > 143 [PSH, ACK] Seq = 1 Ack = 1 Win = 16383 Len = 2
IMAP	CS0	97	251	null	Response: a003 OK [READ-WRITE] SELECT
TCP	CS0	97	348	Note	[TCP Retransmission] 143 > 22592 [ACK] Seq = 1 Ack = 22 Win = 16383 Len = 41
TCP	CS0	64	412	Chat	62762 > 56430 [SYN] Seq = 0 Win = 16383 Len = 0 MSS = 9158 WS = 1

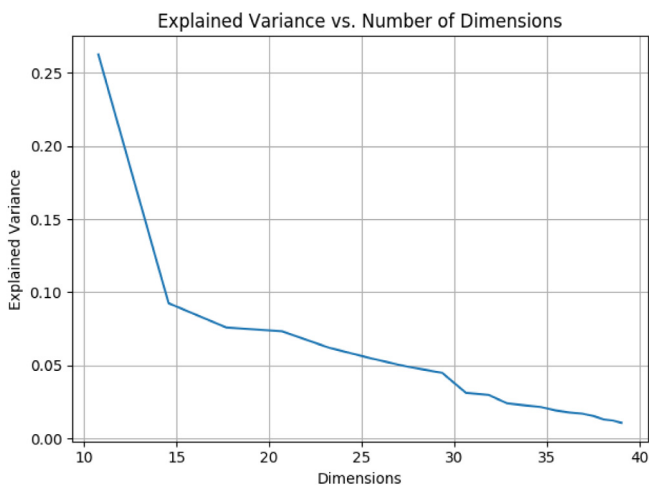


Fig. 6. Explained variance vs. number of dimensions.

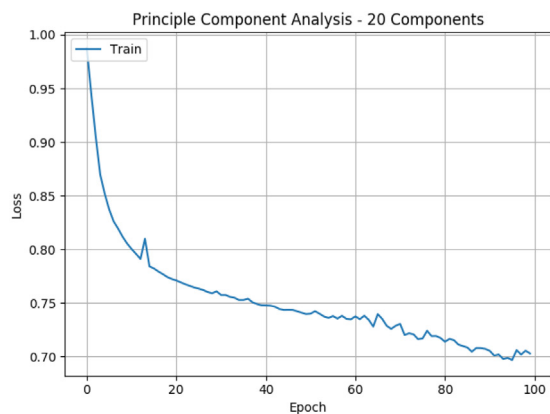
around 15 dimensions and after 20, and the amount of variance gained from added dimensions declines rapidly. Table 3 lists the first 20 principle components, which total 94.3%.

The data stream (ii) dataset is used to train the baseline representation, as reported in the Listing 1, with the minor difference that 46 features are compressed into 20 LSTM nodes, instead of

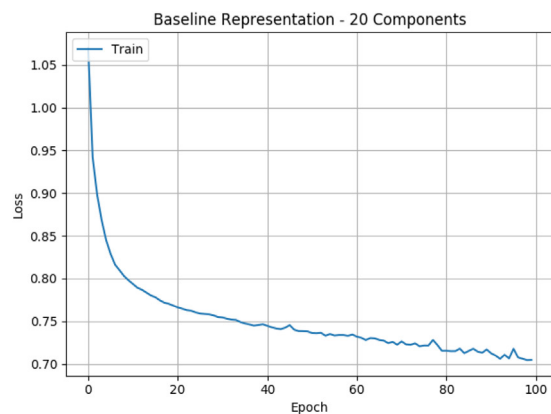
16, to get an accurate comparison. The difference between the number of nodes in the comparison and the model is attributed to the need to maintain the shape of the model. The number of LSTM input nodes correspond both, to the number of dimensions and the number samples that are evaluated before updates. The node size must be chosen such that it properly aligns with the data, while also having enough dimensions to capture as much variance as possible. Therefore, a node size of 16 was chosen for the model because it captured most of the variance while also properly fitting the data. The baseline representation is then executed, and the loss performance of the models *loss* is evaluated and shown in Fig. 7b. The *loss* metric was chosen because when validating autoencoders, it tends to be the best measure of success [31]. Likewise, a separate model was also evaluated using the principle components in Table 3 and is illustrated in Fig. 7a. The two charts seem to be similar, as both after 100 epochs have a *loss* performance of 0.7, approaching zero. This indicates that using an autoencoder to learn the representation of a data stream is equivalent to performing PCA. While AFS is computationally more expensive (more details on its execution time are provided in Section 5), the ability to learn multiple data streams and combine them into a single representation proves to be extremely useful and worthwhile.

4.2. Baseline model performance

The representation is built via the code in Listing 1. In this code, the 46 features are compressed into 16 LSTM nodes, indicated by line 10. The model is then saved, such that it can be



(a) Principle Components in Table 3



(b) Baseline Representation

Fig. 7. PCA comparison of model loss.

```

1 from keras.models import Sequential, load_model
2 from keras.layers import Dense, Dropout, LSTM,
  ↳ RepeatVector, Flatten
3
4 steps = 16
5
6 #Load Model
7 model = load_model('saved_model.h5')
8
9 #Transfer Layers
10 stream_1 = model.layers[0]
11 stream_1.trainable=False
12
13 stream_2 = model.layers[1]
14 stream_2.trainable=False
15
16 #Semi Supervised Model
17 model2 = Sequential()
18 model2.add(stream_1)
19 model2.add(RepeatVector(steps))
20
21 model2.add(stream_2)
22 model2.add(RepeatVector(steps))
23
24 model2.add(LSTM(16, activation='relu',
  ↳ return_sequences=True))
25 model2.add(Dropout(0.5))
26 model2.add(Flatten())
27
28 model2.add(Dense(10, activation='relu'))
29 model2.add(Dense(32, activation='softmax'))
30 model2.compile(loss='categorical_crossentropy',
  ↳ optimizer='adam', metrics=['accuracy'])
31
32 #Fit & Save Model
33 model2.fit(x_train, y_train, epochs=1000, verbose=1)
34 model2.save('new_model.h5')
35

```

Listing 2: Defining Baseline Model

loaded and re-used during the training of future data streams. A new model is created and the LSTM layer from the saved model is transferred. The model is trained with the PCAP file using the same process, making sure that the shapes of both streams (time-step and feature size) align properly. The time-step is the number of samples to evaluate before updating the training weights, and the feature size is the number of features in the data set. After training, the model with the inclusion of the PCAP dataset is again saved. At this point, we have a model with two hidden layers that have been pre-trained, each on a different dataset. Listing 2 shows the code snippet that has been used to configure the baseline model, where the hidden layers transferred from previous models are frozen, shown in line 11 and line 14. This model is then trained and validated with the datasets “UNSW_NB15_training-set.csv” and “UNSW_NB15_testing-set.csv”, respectively. These datasets incorporate all the data that passes through the artifact including PCAP, BRO, and Argus. The training and testing data had an input shape of (16 time-steps, 46 features) with 5,141 samples, amounting to 82,256 data elements. There was a total of 10 training executions for this dataset with 1000 epochs per execution. The number of training executions and epochs were chosen based on a trial and error approach of executing until no further improvements were made. Each execution lasted approximately 25 min running on a Macintosh Apple PC with 4–2.6 GHz CPU cores, 16 GB of memory, and 1 TB of available disk space. The initial observations showed that the *loss* converged to 15% around the 800 epoch mark, and did not improve further. The *accuracy* climbed to 73.2% around the same time period and did not improve going forward. Further review of the data, revealed that there was an imbalance in the prediction class, which caused the

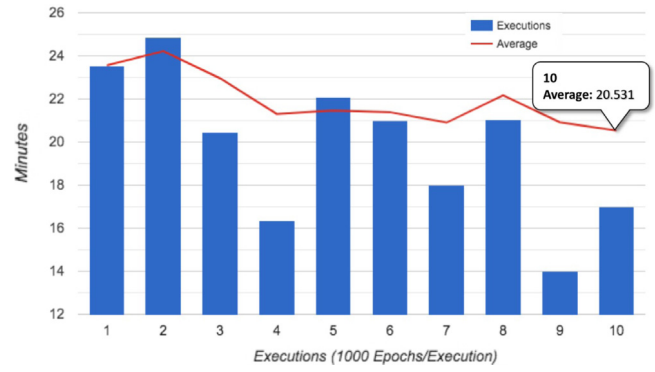


Fig. 8. Data stream training times.

“normal” label classes drowning out the “attack” classes. This was solved by up-sampling the “attack” class, such that both classes ended up with the same number of samples. After repeating the process of re-training the models, the *loss* reached below 10%, and the *accuracy* improved to 81%. Table 4 contains the features that were identified upon a successful classification of an attack, along with their frequency, value, and a confidence score. These interesting features were cross-checked and validated against the “UNSW-NB15_GT.csv” file.

$$\sigma = 1 - \frac{y - y_i}{y} \quad (4)$$

The Confidence Score is calculated as shown in Eq. (4), where y is the actual number of times the feature was used in an attack, and y_i is the number of times the baseline model detected it.

4.3. Hyper compression

The previously defined Algorithm 1 has the potential to capture and learn a wide range of data streams and successfully compress them into a single cohesive stream via the learned data representation. In order to chain representations of multiple data streams, the issue of shape uniformity discussed in Section 3.1.2 adds complexity due to the need for padding. An example of this situation was the network traffic data having 46 desirable attributes, while the packet capture data is having only 12. The packet capture data was padded with seven additional attributes with null values, creating a sparse data set. As a result of the encoding process, the 12 dimensions of the packet capture data are mapped to a four-dimensional space, which causes hyper-compression of the data. Mapping from a high dimensional to low dimensional space is equivalent to performing Principal Component Analysis, which builds new linear attributes from the original ones while maintaining the variance in the data. However, during this process, some information is lost. Therefore, the danger with hyper-compression is that the severe difference between data stream attributes sizes causes re-mappings of high to low dimensions, which in turn leads to information loss. This loss negatively affects the ability of the decoder to reconstruct the input. The solution is to alleviate the need for padding by joining smaller dimensional data streams into a single dataset which can better match the attribute size of other data streams, thus reducing the occurrence of hyper-compression.

5. Observations

The experimental results indicate that using an autoencoder to learn the representation of a single data stream is equivalent to performing PCA. Autoencoders exhibit PCA behavior inherently,

Table 2
BRO & Argus data example—data stream (ii).

BRO & Argus data											
sport	dsport	proto	state	dur	sbytes	dbytes	sttl	dttl	sloss	dloss	service
1390	53	udp	CON	0.0010550000000000002	132	164	31	29	0	0	dns
33661	1024	udp	CON	0.036133	528	304	31	29	0	0	–
1464	53	udp	CON	0.001119	146	178	31	29	0	0	dns
3593	53	udp	CON	0.001209	132	164	31	29	0	0	dns
49664	53	udp	CON	0.001169	146	178	31	29	0	0	dns
Sload	Dload	Spkts	Dpkts	swin	dwin	stcpb	dtcpb	smeansz	dmeansz	trans_depth	res_bdy_len
500473.9375	621800.9375	2	2	0	0	0	0	66	82	0	0
87676.08594	50480.17188	4	4	0	0	0	0	132	76	0	0
521894.5313	636282.375	2	2	0	0	0	0	73	89	0	0
436724.5625	542597.1875	2	2	0	0	0	0	66	82	0	0
499572.25	609067.5625	2	2	0	0	0	0	73	89	0	0
Sjit	Djit	Stime	Ltime	Sintpkt	Dintpkt	tcprtt	synack	ackdat	is_sm_ips_ports	ct_state_ttl	ct_flw_http_mthd
0.0	0.0	1421927414	1421927414	0.017	0.0130000000000000001	0.0	0.0	0.0	0	0	0.0
9.8910100000000001	10.682733	1421927414	1421927414	7.005	7.564333	0.0	0.0	0.0	0	0	0.0
0.0	0.0	1421927414	1421927414	0.017	0.0130000000000000001	0.0	0.0	0.0	0	0	0.0
0.0	0.0	1421927414	1421927414	0.043	0.0139999999999999999	0.0	0.0	0.0	0	0	0.0
0.0	0.0	1421927414	1421927414	0.005	0.003	0.0	0.0	0.0	0	0	0.0
is_ftp_login	ct_ftp_cmd	ct_srv_src	ct_srv_dst	ct_dst_ltm	ct_src_ltm	ct_src_dport_ltm	ct_dst_sport_ltm	ct_dst_src_ltm	attack_cat	–	–
0.0	0	3	7	1	3	1	1	1	0.0	–	–
0.0	0	2	4	2	3	1	1	2	0.0	–	–
0.0	0	12	8	1	2	2	1	1	0.0	–	–
0.0	0	6	9	1	1	1	1	1	0.0	–	–
0.0	0	7	9	1	1	1	1	1	0.0	–	–

regardless of what type of encoder is used. Creating a representation tends to be more computationally expensive than doing PCA. However, by chaining the representations of multiple data streams, each one contributing to a common end, much more information is encoded. This ability has the potential to build flexible, re-usable knowledge structures.

The downside to this great potential is the amount of time needed to create a baseline model. A model training on a 500 MB time series dataset on a machine with 16-core CPUs, and 20 GB of memory, when timed and averaged of 10 separate executions, could exceed an upwards of 20 min as shown in Fig. 8. Thus, considering that for a real system we would have to process multiple data streams, the total training time to create a baseline representation could increase, and potentially become in the order of hours or days. One option is to continuously train a separate baseline model with rolling data streams, while another is operational. Once the training period of a baseline model is complete, it exchanges with the operational model. The previously operational model now begins a new training period. In this fashion, there will always be two baseline models working in tandem. This training approach has yet to be tested on a live system, which the authors plan to validate in future work.

The inclusion of a knowledge engine not only helps to provide the necessary mapping between the input feature and the attack state of the baseline model, but also introduces the framework for self-regulating behavior. In autonomic systems, the desired system behavior is derived from both, knowledge representation and architecture. If the architecture remains static, then changing of the knowledge will also change the behavior. The Python knowledge engine has both a long-term and short-term working memory in forms of fact and knowledge bases. As a consequence, the more prolonged the baseline model executes the more knowledge it acquires and the faster it re-trains.

6. Discussion and future work

Currently, the functionality of AFS is being simulated using public data, but the real potential is demonstrated when running on a live system and collecting real-time data. The ability to adjust the level of granularity and the variety of the data, coupled with on-line training for the baseline model, would allow for increased flexibility and fine-tuning of the learning process.

Future work would first include decreasing the time needed to create the initial baseline representation and to streamline the re-training process. The belief is that using live data would provide more agility in both, collecting and training the baseline representation and baseline model. The intention is to deploy and investigate on a live system. Also, there are a few opportunities to expand the Autonomic Feature Selection to include a dynamic & integrated knowledge repository built on top of the knowledge engine. The knowledge repository must be able to store knowledge structures in any format, such as baseline representations, call graphs, or fuzzy logic rules. The knowledge repository would ensure that all the structures are versioned and fully searchable. Also, storing the metadata along with the knowledge structures would increase the potential for an automated profiling of the artifact. This functionality would allow the knowledge engine to use its inference capabilities to make an actionable decision on how to dynamically update the baseline model.

7. Conclusion

Self-Aware computing systems are systems that exhibit self-regulating behavior and are dependent on interpreting the information from their environments. The amount of data produced by complex systems is superabundant and therefore, it must be

Table 3

PCA with 20 dimensions.

First 20 principle components —data stream (ii)			
0.26253701	0.0925461	0.07599248	0.07341131
0.06217554	0.05498453	0.04906442	0.04492059
0.03128855	0.02989174	0.02416306	0.02280617
0.02164641	0.01931683	0.01794808	0.01715073
0.01545433	0.01317578	0.01236592	0.01086452

Table 4

Interesting features.

Feature	Value	Frequency	Score
Destination port	80 or 8080	15,144	0.77
Protocol	TCP	56,596	0.63
Attack Sub-Category	Browser	2030	0.62
Destination port	179	1260	0.52
Attack Sub-Category	XINETD	24	0.28

reduced to provide useful insights, this process being referred as Feature Selection. To be successful, current methods of feature selection are heavily reliant on two key elements: (1) a well defined and static learning objective and (2) a relevant dataset. These requirements pose an issue for intelligent systems, where there is no guarantee that either of these prerequisites will be available at various needed times and situations. The methods outlined in the literature subscribe to this notion of feature selection and, as a result, they are conducted manually, and relegated to a preprocessing step before performing machine learning.

In this paper two contributions are presented. The first one is represented by the introduction of a detailed taxonomy of feature selection techniques. The second contribution is a novel method to address the limitations inherent in other feature selection methods, called Autonomic Feature Selection (AFS). The taxonomy takes the three primary feature selection classifications, i.e., filter, wrapper, embedded, and breaks them down into subclassifications. The AFS approach follows the Monitor, Analyze, Plan, Execute, and Knowledge (MAPE-K) architecture, and provides a solution to the manual process of feature selection, while identifying the limitations observed in dynamic autonomic systems with many features. The AFS extends an existing autonomic system, the Autonomic Security Management System (ASMS), whose functionality is meant to be an improvement of stand alone Intrusion Response Systems (IRS). The proposed solution describes a two-step process: (1) the deployment of a model to learn the baseline (“normal”) behavior of a system component, and (2) the evaluation of the activity of that component with a report of any features which deviate from that model.

The proposed AFS utilizes a deep learning method called semi-supervised learning to build a baseline representation, which signifies the normal behavior of an artifact. The technique involved uses a combination of representation & transfer learning to train on un-labeled data collected during regular operation of the artifact, and then transferred that knowledge to another model for classification. The new baseline model is trained on labeled data generated while under attack, and learns to distinguish if the artifact is running normally or under attack. The Python Knowledge Engine (Pyke) is deployed with the baseline model to complete the MAPE-K feedback loop. Pyke integrates a form of Logic Programming by providing both, forward-chaining (data-driven) and backward-chaining (goal-directed) inferencing. Also, it automatically generates programs by assembling individual functions into complete call graphs. It utilizes three types of knowledge-bases in tandem, a fact base, a rule base, and a question base. Once operational, the knowledge engine takes notice of the incoming data stream, its features and their values, and adds that information to the fact base via forward-chaining

rules. Then, the backward-chaining rules invoke the baseline model on the saved facts to determine if the artifact is under attack, and the output is persisted back into the fact base for retrieval. If the model determines that an attack is occurring, the list of appropriate ‘interesting features’ are communicated to the manager.

The experimental results and observations show that using an autoencoder to learn the representation of a data stream is equivalent to performing Principle Component Analysis (PCA), while being computationally more expensive. Furthermore, while the ability to learn multiple data streams and combine them into a single representation is extremely useful and worthwhile, the performance of the autoencoder is negatively affected by padding and hyper-compression. However, the training time can be heavily reduced by including the creation of the baseline model as part of the artifact bootstrapping process. Furthermore, parallel computing is also an option to use for either creating the baseline model or using two models in tandem: while one is being trained, the other is deployed and fully operational.

Looking forward to future work, we see opportunities for improvement when running on a live machine and collecting real-time data. Such improvements include on-line training of the baseline model, dynamic & integrated knowledge repository, and automated artifact profiling.

In conclusion, the contributions presented herein provide a foundation for shifting feature selection away from being a manual preprocessing step that requires a well defined static learning objective and a relevant dataset, to being a genuinely autonomic component that is fully integrated into the learning process. This opens up the avenues for these systems to become self-aware.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] R.E. Bellman, *Dynamic Programming*, Princeton University Press, 1957.
- [2] M.R. Berthold, C. Borgelt, F. Höppner, F. Klawonn, *Guide to Intelligent Data Analysis*, Springer, 2010.
- [3] J. Reunanen, Overfitting in making comparisons between variable selection methods, *JMLR* 3 (2003) 1371–1782.
- [4] M. Dash, H. Liu, Feature selection for classification, *Intell. Data Anal.* 1 (3) (1997) 131–156.
- [5] L. Yu, H. Liu, Feature selection for high-dimensional data: A fast correlation-based filter solution, in: *International Conference on Machine Learning*, 2003.
- [6] K. Kira, L.A. Rendell, A practical approach to feature selection, *Mach. Learn. Proc.* (1992) 249–256.
- [7] I. Guyon, A. Elisseeff, An introduction to variable and feature selection, *J. Mach. Learn. Res.* (2003).
- [8] J. Li, H. Liu, Challenges of feature selection for big data analytics, *IEEE Intell. Syst.* (2017).
- [9] P. Lalanda, J.A. McCann, A. Diaconescu, *Autonomic Computing: Principles, Design, and Implementation*, Springer, 2014.
- [10] S. Iannucci, S. Abdelwahed, A probabilistic approach to autonomic security management, in: *IEEE International Conference on Autonomic Computing (ICAC)*, 2016.
- [11] S. Iannucci, S. Abdelwahed, Model-based response planning strategies for autonomic intrusion protection, *ACM Trans. Auton. Adapt. Syst. (TAAS)* 13 (1) (2018) 4.
- [12] S.J. Pan, Q. Yang, A survey on transfer learning, *IEEE Trans. Knowl. Data Eng.* (2009).
- [13] J. Lu, V. Behbood, P. Hao, H. Zuo, S. Xue, G. Zhang, Transfer learning using computational intelligence: A survey, *ScienceDirect Knowl.-Based Syst.* 80 (2015) 14–23.
- [14] R. Murch, *Autonomic Computing*, IBM Press, 2004.
- [15] A.P. Engelbrecht, *Computational Intelligence, An Introduction*, second ed., John Wiley & Sons Inc., 2007.
- [16] G. Chandrashekar, F. Sahin, A survey on feature selection methods, *Comput. Electr. Eng.* (2014).
- [17] Y. Saeyns, I. Inza, P. Larranga, A review of feature selection techniques in bioinformatics, *bioinformatics* 23 (19) (2007) 2507–2517.
- [18] X. Jin, A. Xu, R. Bie, P. Guo, Machine learning techniques and chi-square feature selection for cancer classification using sage gene expression profiles, in: *International Workshop on Data Mining for Biomedical Applications*, 2006, pp. 106–115.
- [19] Q. Gu, Z. Li, J. Han, Generalized Fisher Score for Feature Selection, Cornell University, 2012, arXiv.org (arXiv:1202.3725).
- [20] P.S. Bradley, O.L. Mangasarian, W.N. Street, Feature selection via mathematical programming, *INFORMS J. Comput.* 10 (2) (1998) 209–217.
- [21] D.C. Montgomery, G.C. Runger, *Applied Statistics and Probability for Engineers*, third ed., John Wiley & Sons, Inc., 2003.
- [22] C.-L. Huang, C.-J. Wang, A ga-based feature selection and parameters optimization for support vector machines, *Expert Syst. Appl.* (2006).
- [23] K. Yan, D. Zhang, Feature selection and analysis on correlated gas sensor data with recursive feature elimination, *Sensors Actuators B* (2015).
- [24] D.W. Aha, R.L. Bankert, A comparative evaluation of sequential feature selection algorithms, *Learn. Data* (1996).
- [25] K.F. Patrenahalli, M. Narendra, A branch and bound algorithm for feature subset selection, *IEEE Trans. Comput. C-26* (9) (1977).
- [26] G. Stein, B. Chen, A.S. Wu, K.A. Hua, Decision tree classifier for network intrusion detection with ga-based feature selection, in: *Proceedings of the 43rd Annual Southeast Regional Conference*, Vol. 2, ACM, 2005, pp. 136–141.
- [27] K. jae Kim, I. Han, Genetic algorithms approach to feature discretization in artificial neural networks for the prediction of stock price index, *Expert Syst. Appl.* 19 (2000) 125–132.
- [28] F.Z. Brill, D.E. Brown, W.N. Martin, Fast generic selection of features for neural network classifiers, *IEEE Trans. Neural Netw.* 3 (2) (1992) 324–328.
- [29] K.O. Stanley, R. Miikkulainen, Evolving neural networks through augmenting topologies, *Evol. Comput.* 10 (2) (2002) 99–127.
- [30] Y. Bengio, A. Courville, P. Vincent, *Representation learning: A review and new perspectives*, 2014.
- [31] I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, The MIT Press, 2016.
- [32] Y. Bengio, *Learning Deep Architectures for AI*, in: *Foundations and Trends in Machine Learning*, vol. 2, now Publishers Inc., 2009.
- [33] Y. Kim, H. Lee, E.M. Provost, Deep learning for robust feature generation in audiovisual emotion recognition, in: *IEEE International Conference on Acoustics, Speech and Signal Processing*, 2013.
- [34] B. Babic, N. Nesić, Z. Miljković, A review of automated feature recognition with rule-based pattern recognition, *Comput. Ind.* 59 (2008) 321–337.
- [35] F. Muge, I. Granado, M. Mengucci, P. Pina, V. Ramos, N. Sirakov, J. Caldas, P. Pinto, A. Marcolino, M. Ramalho, P. Vieira, A.M. do Amaral, Automatic feature extraction and recognition for digital access of books of the renaissance, *Res. Adv. Technol. Digit. Lib.* (2000) 1–13.
- [36] F. Nie, H. Huang, X. Cai, C.H. Ding, Efficient and robust feature selection via joint $\ell_{2,1}$ -norms minimization, in: *Advances in Neural Information Processing Systems* 23, 2010.
- [37] T. Hastie, R. Tibshirani, J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, second ed., Springer, 2009.
- [38] S. Perkins, K. Lacker, J. Theiler, Grafting: Fast incremental feature selection by gradient descent in function space, *J. Mach. Learn. Res.* 3 (2003) 1333–1356.
- [39] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [40] H. Lee, Y. Largman, P. Pham, A.Y. Ng, Unsupervised feature learning for audio classification using convolutional deep belief networks, *Adv. Neural Inf. Process. Syst.* (2009).
- [41] K. Noda, Y. Yamaguchi, K. Nakadai, H.G. Okuno, T. Ogata, Audio-visual speech recognition using deep learning, *Appl. Intell.* 42 (4) (2015) 722–737.
- [42] Y. Kim, H. Lee, E.M. Provost, Deep learning for robust feature generation in audiovisual emotion recognition, in: *IEEE International Conference on Acoustics, Speech and Signal Processing*, 2013.
- [43] R. Sarikaya, G.E. Hinton, A. Deoras, Application of deep belief networks for natural language understanding, *IEEE/ACM Trans. Audio Speech Lang. Process. (TASLP)* 22 (4) (2014) 778–784.
- [44] T. Young, D. Hazarika, S. Poria, E. Cambria, Recent trends in deep learning based natural language processing, *IEEE Comput. Intell. Mag.* 13 (3) (2018) 55–75.
- [45] R. Socher, C.C.-Y. Lin, A.Y. Ng, C.D. Manning, Parsing natural scenes and natural language with recursive neural networks, in: *International Conference on Machine Learning*, Vol. 28, 2011.

- [46] D. Rumelhart, G. Hinton, R. Williams, Learning representation by back-propagating errors, *Nature* (323) (1986) 533–536.
- [47] Python Knowledge Engine, URL <http://pyke.sourceforge.net/index.html>.
- [48] J.S. Nour Moustafa, [dataset] UNSW-NB15, URL <https://www.unsw.adfa.edu.au/unsw-canberra-cyber/cybersecurity/ADFA-NB15-Datasets/>.
- [49] N. Moustafa, A. Moustafa, Designing an Online and Reliable Statistical Anomaly Detection Framework for Dealing with Large High-Speed Network Traffic (Ph.D. thesis), School of Engineering and Information Technology, The University of New South Wales, Australia, 2017.
- [50] J.S. Nour Moustafa, Generalized outlier Gaussian mixture technique based on automated association features for simulating and detecting web application attacks, *IEEE Trans. Sustain. Comput.* (2018).
- [51] J.S. Nour Moustafa, An ensemble intrusion detection technique based on proposed statistical flow features for protecting network traffic of internet of things., *IEEE Internet Things J.* (2018).
- [52] J.S. Nour Moustafa, Towards developing network forensic mechanism for botnet activities in the iot based on machine learning techniques, in: *International Conference on Mobile Networks and Management*, 2017.
- [53] J.S. Nour Moustafa, Privacy preservation intrusion detection technique for scada systems, in: *Military Communications and Information Systems Conference (MilCIS)*, 2017.

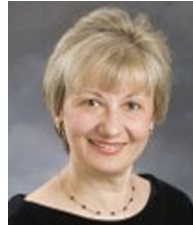


Patrick Day received M.S. degrees in both Applied Systems Engineering and Computer Science from the Georgia Institute of Technology, Atlanta, GA, United States. He is currently pursuing his Ph.D. degree in Computational Engineering with the Department of Computer Science and Engineering at Mississippi State University, Starkville, MS, United States. His main research interests include autonomic computing, computational intelligence, machine learning, and software engineering & architecture. In addition, he has more than 15 years of industry experience with such companies as GE, ADP, The Weather Company (IBM), ABB, Oracle, and Hewlett Packard.



Computing.

Stefano Iannucci is an Assistant Professor of Computer Science and Engineering at Mississippi State University. He received his Ph.D. in 2015 from the University of Rome Tor Vergata. His research focuses on cybersecurity automation, autonomic computing, Internet of things and performance modeling and benchmarking. He published over 20 papers in top journals and conferences. Dr. Iannucci has chaired several international workshops and has been the workshops chair for IEEE ICCAC, one of the leading conferences in autonomic computing. He is Associate Editor of Springer Cluster



Ioana Banicescu is a professor in the Department of Computer Science and Engineering at Mississippi State University (MSU). Between 2009 and 2017, she was also a Director of the Center for Cloud and Autonomic Computing at MSU, and also a Co-Director of the National Science Foundation Center for Cloud and Autonomic Computing. She received the Diploma in Engineering (Electronics and telecommunications) from Polytechnic University of Bucharest, and the M.S. and the Ph.D. degrees in Computer Science from New York University - Polytechnic Institute. Ioana's research interests include parallel algorithms, scientific computing, scheduling theory, load balancing algorithms, performance modeling, analysis and prediction. Currently, her research focus is on autonomic computing, performance optimization for problems in computational science, and graph analytics. She has given many invited talks at universities, government laboratories, and at various national and international forums in the United States and overseas. Ioana is the recipient of a number of awards for research and scholarship from the National Science Foundation (NSF). She served and continues to serve on numerous research review panels for advanced research grants in the US and Europe, on steering and program committees of a number of international ACM and IEEE conferences, symposia and workshops, and on the Executive Board and Advisory Board of the IEEE technical Committee on Parallel Processing (CPP). Ioana was an Associate Editor of the Cluster Computing journal and the International Journal on Computational Science and Engineering. Over the years, she was recognized with many distinctions for her scholarly contributions.